



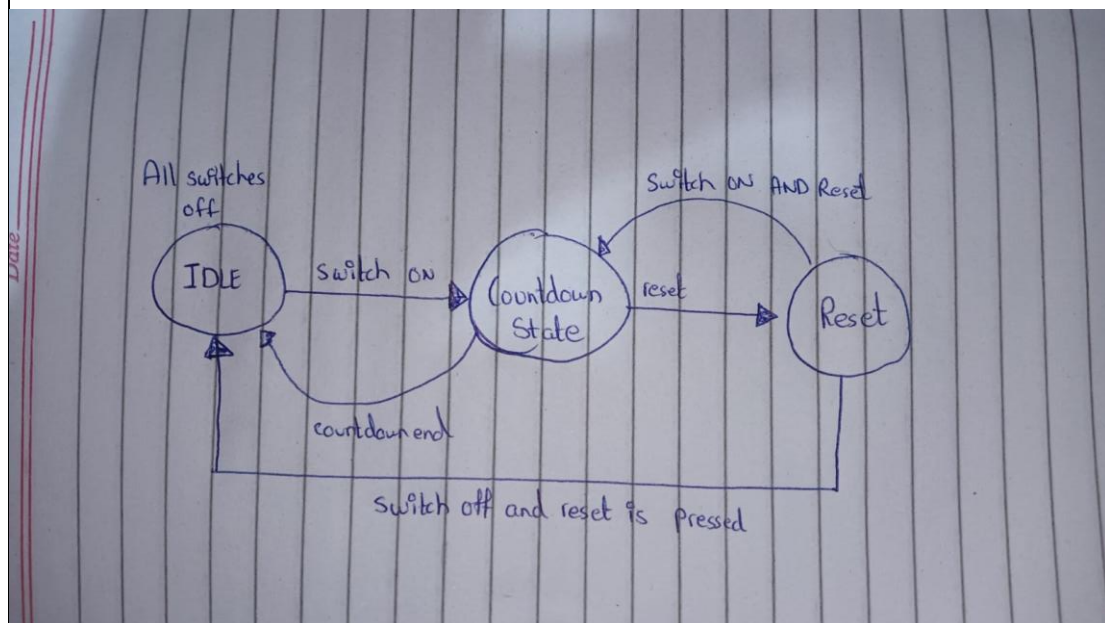
## Lab 05 – Worksheet

|                                                        |                    |              |
|--------------------------------------------------------|--------------------|--------------|
| Name: Mohammad Husain Naqvi<br>Syed Fakhar Iqbal Rizvi | ID: 11062<br>11091 | Section: T-4 |
|--------------------------------------------------------|--------------------|--------------|

**Note:** Assumptions and logics should be explained separately in tasks after the task results.

### Task 1

FSM:





### Comments

## Task 2

### Modules + Testbench:

#### TOP MODULE:

```
1  `timescale 1ns/1ps
2
3  module tb_top;
4      reg clk;
5      reg rst;
6      reg tick;
7      reg [3:0] start_value;
8      reg start_valid;
9      wire [3:0] display_value;
10     wire busy;
11
12     //we use the FSM counter to instantiate
13     fsm_counter uut (
14         .clk(clk),
15         .rst(rst),
16         .tick(tick),
17         .start_value(start_value),
18         .start_valid(start_valid),
19         .display_value(display_value),
20         .busy(busy)
21     );
22
23     //generation of clock
24     initial begin //this is where we generate the clock
25         clk = 0;
26         forever #5 clk = ~clk; //keeping a 10ns period
27     end
28
29     //-----
```



```
        clk = 0;
        forever #5 clk = ~clk; //keeping a 10ns period
    end

    //generation of 1 sec ticks
    initial begin
        tick = 0;
        forever begin
            #50 tick = 1; // 1 sec
            #10 tick = 0;
        end
    end

    initial begin
        //Initial IDLE STATE
        rst = 1; //Reset is pressed to ensure we are in our idle state
        start_valid = 0;
        start_value = 0;
        #20;
        rst = 0;

        //IF SWITCH 8 IS PRESSED
        #20;
        start_value = 4'd8;
        start_valid = 1; //need to tell if we have pressed the switch or not
        #10;
        start_valid = 0;
        #1000; //full countdown time

        start_value = 4'd5;
        start_valid = 1;
        #10;
        start_valid = 0;
        #100;
        rst = 1; //during the cntdwn reset
        #20;
        rst = 0;
        #200;
        $stop;
    end

    //show values
    always @(posedge tick) begin
        if (busy)
            $display("Time %t : Display = %d", $time, display_value);
    end
endmodule
```

**De-bouncer:**

```
module debouncer(  
    input clk,  
    input pbin,  
    output reg pbout  
);  
  
    reg [19:0] counter = 0;  
    reg sync_0 = 0;  
    reg sync_1 = 0;  
  
    always @(posedge clk) begin  
        sync_0 <= pbin;  
        sync_1 <= sync_0;  
    end  
  
    always @(posedge clk) begin  
        if (pbout == sync_1)  
            counter <= 0;  
        else begin  
            counter <= counter + 1;  
            if (counter == 1_000_000) begin  
                pbout <= sync_1;  
                counter <= 0;  
            end  
        end  
    end  
end  
endmodule
```

```
module debouncer(  
    input clk,  
    input pbin,  
    output reg pbout  
);
```

```
    reg [19:0] counter = 0;  
    reg sync_0 = 0;  
    reg sync_1 = 0;
```

```
    always @(posedge clk) begin  
        sync_0 <= pbin;  
        sync_1 <= sync_0;  
    end
```

```
    always @(posedge clk) begin  
        if (pbout == sync_1)  
            counter <= 0;  
        else begin  
            counter <= counter + 1;  
            if (counter == 1_000_000) begin
```



```
        pbout <= sync_1;
        counter <= 0;
    end
end
end

endmodule
```

### Priority encoder:

```
module priority_encoder(
    input [15:0] sw,
    output reg [3:0] value,
    output reg valid
);

    integer i;

    always @(*) begin
        value = 0;
        valid = 0;

        for (i = 15; i >= 0; i = i - 1) begin
            if (sw[i]) begin
                value = i[3:0];
                valid = 1;
            end
        end
    end
end

endmodule
```

```
module priority_encoder(
    input [15:0] sw,
    output reg [3:0] value,
    output reg valid
);
```

```
    integer i;
```

```
    always @(*) begin
        value = 0;
        valid = 0;
```

```
        for (i = 15; i >= 0; i = i - 1) begin
            if (sw[i]) begin
                value = i[3:0];
                valid = 1;
            end
        end
    end
end
```

```
endmodule
```

**Clock Divider:**

```
module clock_divider(  
    input clk,  
    input rst,  
    output reg tick  
);  
  
    reg [26:0] count;  
  
    always @(posedge clk or posedge rst) begin  
        if (rst) begin  
            count <= 0;  
            tick <= 0;  
        end  
        else if (count == 100_000_000 - 1) begin  
            count <= 0;  
            tick <= 1;  
        end  
        else begin  
            count <= count + 1;  
            tick <= 0;  
        end  
    end  
end  
  
endmodule
```

```
module clock_divider(  
    input clk,  
    input rst,  
    output reg tick  
);
```

```
    reg [26:0] count;
```

```
    always @(posedge clk or posedge rst) begin  
        if (rst) begin  
            count <= 0;  
            tick <= 0;  
        end  
        else if (count == 100_000_000 - 1) begin  
            count <= 0;  
            tick <= 1;  
        end  
        else begin  
            count <= count + 1;  
            tick <= 0;  
        end  
    end  
end  
  
endmodule
```



## FSM\_counter:

```
1 module fsm_counter(  
2     input clk,  
3     input rst,  
4     input tick,  
5     input [3:0] start_value, //starting value to display  
6     input start_valid,  
7     output reg [3:0] display_value, //displaying current value on the seven seg  
8     output reg busy //if its counting dwn, we need to know that  
9 );  
10  
11 localparam IDLE = 2'b00; //IDLE STATE 00  
12 localparam WAIT1 = 2'b01; //wait a second before counting down  
13 localparam COUNT = 2'b10;  
14  
15 reg [1:0] state;  
16 reg [3:0] counter;  
17 reg start_valid_d;  
18  
19 always @(posedge clk or posedge rst) begin  
20     if (rst)  
21         start_valid_d <= 0;  
22     else  
23         start_valid_d <= start_valid;  
24 end  
25  
26 wire start_pulse = start_valid & ~start_valid_d;  
27  
28 always @(posedge clk or posedge rst) begin  
29     if (rst) begin  
30         state <= IDLE;  
31         counter <= 0;  
32  
33         display_value <= 0;  
34         busy <= 0;  
35     end  
36     else begin  
37         case (state)  
38             IDLE: begin  
39                 busy <= 0;  
40                 if (start_pulse) begin  
41                     counter <= start_value;  
42                     display_value <= start_value;  
43                     busy <= 1;  
44                     state <= WAIT1; //waiting for one full tick  
45                 end  
46             end  
47             WAIT1: begin  
48                 busy <= 1;  
49                 if (tick)  
50                     state <= COUNT; //go to the next number after 1 sec  
51             end  
52             COUNT: begin  
53                 busy <= 1;  
54                 if (tick) begin  
55                     if (counter > 0) begin  
56                         counter <= counter - 1;  
57                         display_value <= counter - 1;  
58                     end  
59                     else begin  
60                         state <= IDLE;  
61                         counter <= 0;  
62                         display_value <= 0;  
63                         busy <= 0;  
64                     end  
65                 end  
66             end  
67         endcase  
68     end  
69 end
```



```
        state <= IDLE;
        busy <= 0;
    end
end
end
endcase
end
end
endmodule
```

```
module fsm_counter(
    input clk,
    input rst,
    input tick,
    input [3:0] start_value, //starting value to display
    input start_valid,
    output reg [3:0] display_value, //displaying current value on the seven seg
    output reg busy //if its counting dwn, we need to know that
);

localparam IDLE = 2'b00; //IDLE STATE 00
localparam WAIT1 = 2'b01; //wait a second before counting down
localparam COUNT = 2'b10;

reg [1:0] state;
reg [3:0] counter;
reg start_valid_d;

always @(posedge clk or posedge rst) begin
    if (rst)
        start_valid_d <= 0;
    else
        start_valid_d <= start_valid;
    end

    wire start_pulse = start_valid & ~start_valid_d;

    always @(posedge clk or posedge rst) begin
        if (rst) begin
            state <= IDLE;
            counter <= 0;
            display_value <= 0;
            busy <= 0;
        end
    end
end
```



```
end
else begin
  case (state)
    IDLE: begin
      busy <= 0;
      if (start_pulse) begin
        counter <= start_value;
        display_value <= start_value;
        busy <= 1;
        state <= WAIT1; //waiting for one full tick
      end
    end
  end

  WAIT1: begin
    busy <= 1;
    if (tick)
      state <= COUNT; //go to the next number after 1 sec
    end

    COUNT: begin
      busy <= 1;
      if (tick) begin
        if (counter > 0) begin
          counter <= counter - 1;
          display_value <= counter - 1;
        end
        else begin
          state <= IDLE;
          busy <= 0;
        end
      end
    end
  end
end

endcase
end
end

endmodule
```

**LED Module:**

```
1 module leds(  
2     input clk,  
3     input rst,  
4     input [15:0] btns,  
5     input [31:0] writeData,  
6     input writeEnable,  
7     input readEnable,  
8     input [29:0] memAddress,  
9     input [3:0] display_value, //read the current value from FSM  
10    output reg [31:0] readData,  
11    output [6:0] seg,  
12    output [3:0] an  
13 );  
14  
15 always @(posedge clk or posedge rst) begin  
16     if (rst)  
17         readData <= 0;  
18     else if (readEnable)  
19         readData <= {28'b0, display_value};  
20     else  
21         readData <= 0;  
22 end  
23  
24 sevenseg_decode ssd( //seven segment decoder initialized  
25     .bin(display_value),  
26     .seg(seg)  
27 );  
28 //we are using only the first segment  
29 assign an = 4'b1110;  
30 endmodule
```

```
module leds(  
    input clk,  
    input rst,  
    input [15:0] btns,  
    input [31:0] writeData,  
    input writeEnable,  
    input readEnable,  
    input [29:0] memAddress,  
    input [3:0] display_value, //read the current value from FSM  
    output reg [31:0] readData,  
    output [6:0] seg,  
    output [3:0] an  
);
```

```
always @(posedge clk or posedge rst) begin  
    if (rst)  
        readData <= 0;  
    else if (readEnable)  
        readData <= {28'b0, display_value};  
    else  
        readData <= 0;  
end
```



```
sevensseg_decode ssd( //seven segment decoder initialized
    .bin(display_value),
    .seg(seg)
);
//we are using only the first segment
assign an = 4'b1110;
endmodule
```

### Seven segment :

```
1  `timescale 1ns/1ps
2  module sevensseg_decode(
3      input wire [3:0] bin,
4      output reg [6:0] seg
5  );
6
7      always @(*) begin
8          case (bin)
9              4'h0: seg = 7'b1000000;
10             4'h1: seg = 7'b1111001;
11             4'h2: seg = 7'b0100100;
12             4'h3: seg = 7'b0110000;
13             4'h4: seg = 7'b0011001;
14             4'h5: seg = 7'b0010010;
15             4'h6: seg = 7'b0000010;
16             4'h7: seg = 7'b1111000;
17             4'h8: seg = 7'b0000000;
18             4'h9: seg = 7'b0010000;
19             4'hA: seg = 7'b0001000;
20             4'hB: seg = 7'b0000011;
21             4'hC: seg = 7'b1000110;
22             4'hD: seg = 7'b0100001;
23             4'hE: seg = 7'b0000110;
24             4'hF: seg = 7'b0001110;
25             default: seg = 7'b1111111;
26         endcase
27     end
28 endmodule
```

```
`timescale 1ns/1ps
module sevensseg_decode(
    input wire [3:0] bin,
    output reg [6:0] seg
);

always @(*) begin
    case (bin)
        4'h0: seg = 7'b1000000;
        4'h1: seg = 7'b1111001;
        4'h2: seg = 7'b0100100;
        4'h3: seg = 7'b0110000;
        4'h4: seg = 7'b0011001;
        4'h5: seg = 7'b0010010;
        4'h6: seg = 7'b0000010;
        4'h7: seg = 7'b1111000;
        4'h8: seg = 7'b0000000;
```



```
4'h9: seg = 7'b0010000;  
4'hA: seg = 7'b0001000;  
4'hB: seg = 7'b0000011;  
4'hC: seg = 7'b1000110;  
4'hD: seg = 7'b0100001;  
4'hE: seg = 7'b0000110;  
4'hF: seg = 7'b0001110;  
default: seg = 7'b1111111;  
endcase  
end  
endmodule
```

### TESTBENCH:

```
1  `timescale 1ns/1ps  
2  
3  module tb_top;  
4      reg clk;  
5      reg rst;  
6      reg tick;  
7      reg [3:0] start_value;  
8      reg start_valid;  
9      wire [3:0] display_value;  
10     wire busy;  
11  
12     //we use the FSM counter to instantiate  
13     fsm_counter uut (  
14         .clk(clk),  
15         .rst(rst),  
16         .tick(tick),  
17         .start_value(start_value),  
18         .start_valid(start_valid),  
19         .display_value(display_value),  
20         .busy(busy)  
21     );  
22  
23     //generation of clock  
24     initial begin //this is where we generate the clock  
25         clk = 0;  
26         forever #5 clk = ~clk; //keeping a 10ns period  
27     end  
28  
29     //generation of 1 sec ticks  
30     initial begin  
31         tick = 0;
```



```
    tick = 0;
    forever begin
        #50 tick = 1; // 1 sec
        #10 tick = 0;
    end
end
initial begin
    //Initial IDLE STATE
    rst = 1; //Reset is pressed to ensure we are in our idle state
    start_valid = 0;
    start_value = 0;
    #20;
    rst = 0;

    #20; //IF SWITCH 8 IS PRESSED
    start_value = 4'd8;
    start_valid = 1; //need to tell if we have pressed the switch or not
    #10;
    start_valid = 0;
    #1000; //full countdown time

    start_value = 4'd5;
    start_valid = 1;
    #10;
    start_valid = 0;
    #100;
    rst = 1; //during the cntdwn reset
    #20;
    rst = 0;

    #200;
    $stop;
end
//show values
always @(posedge tick) begin
    if (busy)
        $display("Time %t : Display = %d", $time, display_value);
end
endmodule
```

```
`timescale 1ns/1ps
```

```
module tb_top;
    reg clk;
    reg rst;
    reg tick;
    reg [3:0] start_value;
    reg start_valid;
    wire [3:0] display_value;
    wire busy;

    //we use the FSM counter to instantiate
    fsm_counter uut (
        .clk(clk),
        .rst(rst),
        .tick(tick),
        .start_value(start_value),
```



```
.start_valid(start_valid),
.display_value(display_value),
.busy(busy)
);

//generation of clock
initial begin //this is where we generate the clock
    clk = 0;
    forever #5 clk = ~clk; //keeping a 10ns period
end

//generation of 1 sec ticks
initial begin
    tick = 0;
    forever begin
        #50 tick = 1; // 1 sec
        #10 tick = 0;
    end
end

initial begin
    //Initial IDLE STATE
    rst = 1; //Reset is pressed to ensure we are in our idle state
    start_valid = 0;
    start_value = 0;
    #20;
    rst = 0;

    //IF SWITCH 8 IS PRESSED
    #20;
    start_value = 4'd8;
    start_valid = 1; //need to tell if we have pressed the switch or not
    #10;
    start_valid = 0;
    #1000; //full countdown time

    start_value = 4'd5;
    start_valid = 1;
    #10;
    start_valid = 0;
    #100;
    rst = 1; //during the cntdwn reset
    #20;
    rst = 0;
    #200;
    $stop;
end
```



```
//show values
always @(posedge tick) begin
    if (busy)
        $display("Time %t : Display = %d", $time, display_value);
    end
endmodule
```

Results:

### Task 3

*XDC file*



```
## =====
## Basys 3 Master XDC
## Signals:
##  sw[15:0]  - 16 switches
##  led[15:0] - 16 LEDs
##  seg[7:0]  - 7-segment segments + decimal point
##  an[3:0]   - 7-segment digit enables
##  clk       - 100 MHz clock
##  rst_btn   - Center push button
## =====

## =====
## CLOCK
## =====

set_property PACKAGE_PIN W5 [get_ports clk]
set_property IOSTANDARD LVCMOS33 [get_ports clk]

create_clock -add -name sys_clk_pin -period 10.00 \
    -waveform {0 5} [get_ports clk]

## =====
## RESET BUTTON
## Basys 3 Center Button
## =====

set_property PACKAGE_PIN U18 [get_ports rst_btn]
set_property IOSTANDARD LVCMOS33 [get_ports rst_btn]

## =====
## SWITCHES
## =====
```



```
set_property PACKAGE_PIN V17 [get_ports {sw[0]}]
set_property IOSTANDARD LVCMOS33 [get_ports {sw[0]}]

set_property PACKAGE_PIN V16 [get_ports {sw[1]}]
set_property IOSTANDARD LVCMOS33 [get_ports {sw[1]}]

set_property PACKAGE_PIN W16 [get_ports {sw[2]}]
set_property IOSTANDARD LVCMOS33 [get_ports {sw[2]}]

set_property PACKAGE_PIN W17 [get_ports {sw[3]}]
set_property IOSTANDARD LVCMOS33 [get_ports {sw[3]}]

set_property PACKAGE_PIN W15 [get_ports {sw[4]}]
set_property IOSTANDARD LVCMOS33 [get_ports {sw[4]}]

set_property PACKAGE_PIN V15 [get_ports {sw[5]}]
set_property IOSTANDARD LVCMOS33 [get_ports {sw[5]}]

set_property PACKAGE_PIN W14 [get_ports {sw[6]}]
set_property IOSTANDARD LVCMOS33 [get_ports {sw[6]}]

set_property PACKAGE_PIN W13 [get_ports {sw[7]}]
set_property IOSTANDARD LVCMOS33 [get_ports {sw[7]}]

set_property PACKAGE_PIN V2 [get_ports {sw[8]}]
set_property IOSTANDARD LVCMOS33 [get_ports {sw[8]}]

set_property PACKAGE_PIN T3 [get_ports {sw[9]}]
set_property IOSTANDARD LVCMOS33 [get_ports {sw[9]}]

set_property PACKAGE_PIN T2 [get_ports {sw[10]}]
set_property IOSTANDARD LVCMOS33 [get_ports {sw[10]}]

set_property PACKAGE_PIN R3 [get_ports {sw[11]}]
```



```
set_property IOSTANDARD LVCMOS33 [get_ports {sw[11]}]
```

```
set_property PACKAGE_PIN W2 [get_ports {sw[12]}]
```

```
set_property IOSTANDARD LVCMOS33 [get_ports {sw[12]}]
```

```
set_property PACKAGE_PIN U1 [get_ports {sw[13]}]
```

```
set_property IOSTANDARD LVCMOS33 [get_ports {sw[13]}]
```

```
set_property PACKAGE_PIN T1 [get_ports {sw[14]}]
```

```
set_property IOSTANDARD LVCMOS33 [get_ports {sw[14]}]
```

```
set_property PACKAGE_PIN R2 [get_ports {sw[15]}]
```

```
set_property IOSTANDARD LVCMOS33 [get_ports {sw[15]}]
```

```
## =====
```

```
## LEDs
```

```
## =====
```

```
set_property PACKAGE_PIN U16 [get_ports {led[0]}]
```

```
set_property IOSTANDARD LVCMOS33 [get_ports {led[0]}]
```

```
set_property PACKAGE_PIN E19 [get_ports {led[1]}]
```

```
set_property IOSTANDARD LVCMOS33 [get_ports {led[1]}]
```

```
set_property PACKAGE_PIN U19 [get_ports {led[2]}]
```

```
set_property IOSTANDARD LVCMOS33 [get_ports {led[2]}]
```

```
set_property PACKAGE_PIN V19 [get_ports {led[3]}]
```

```
set_property IOSTANDARD LVCMOS33 [get_ports {led[3]}]
```

```
set_property PACKAGE_PIN W18 [get_ports {led[4]}]
```

```
set_property IOSTANDARD LVCMOS33 [get_ports {led[4]}]
```

```
set_property PACKAGE_PIN U15 [get_ports {led[5]}]
```



```
set_property IOSTANDARD LVCMOS33 [get_ports {led[5]}]

set_property PACKAGE_PIN U14 [get_ports {led[6]}]
set_property IOSTANDARD LVCMOS33 [get_ports {led[6]}]

set_property PACKAGE_PIN V14 [get_ports {led[7]}]
set_property IOSTANDARD LVCMOS33 [get_ports {led[7]}]

set_property PACKAGE_PIN V13 [get_ports {led[8]}]
set_property IOSTANDARD LVCMOS33 [get_ports {led[8]}]

set_property PACKAGE_PIN V3 [get_ports {led[9]}]
set_property IOSTANDARD LVCMOS33 [get_ports {led[9]}]

set_property PACKAGE_PIN W3 [get_ports {led[10]}]
set_property IOSTANDARD LVCMOS33 [get_ports {led[10]}]

set_property PACKAGE_PIN U3 [get_ports {led[11]}]
set_property IOSTANDARD LVCMOS33 [get_ports {led[11]}]

set_property PACKAGE_PIN P3 [get_ports {led[12]}]
set_property IOSTANDARD LVCMOS33 [get_ports {led[12]}]

set_property PACKAGE_PIN N3 [get_ports {led[13]}]
set_property IOSTANDARD LVCMOS33 [get_ports {led[13]}]

set_property PACKAGE_PIN P1 [get_ports {led[14]}]
set_property IOSTANDARD LVCMOS33 [get_ports {led[14]}]

set_property PACKAGE_PIN L1 [get_ports {led[15]}]
set_property IOSTANDARD LVCMOS33 [get_ports {led[15]}]

## =====
## 7-SEGMENT DISPLAY
```



```
## seg[0] = CA
## seg[1] = CB
## seg[2] = CC
## seg[3] = CD
## seg[4] = CE
## seg[5] = CF
## seg[6] = CG
## seg[7] = DP
## =====

set_property PACKAGE_PIN W7 [get_ports {seg[0]}]
set_property IOSTANDARD LVCMOS33 [get_ports {seg[0]}]

set_property PACKAGE_PIN W6 [get_ports {seg[1]}]
set_property IOSTANDARD LVCMOS33 [get_ports {seg[1]}]

set_property PACKAGE_PIN U8 [get_ports {seg[2]}]
set_property IOSTANDARD LVCMOS33 [get_ports {seg[2]}]

set_property PACKAGE_PIN V8 [get_ports {seg[3]}]
set_property IOSTANDARD LVCMOS33 [get_ports {seg[3]}]

set_property PACKAGE_PIN U5 [get_ports {seg[4]}]
set_property IOSTANDARD LVCMOS33 [get_ports {seg[4]}]

set_property PACKAGE_PIN V5 [get_ports {seg[5]}]
set_property IOSTANDARD LVCMOS33 [get_ports {seg[5]}]

set_property PACKAGE_PIN U7 [get_ports {seg[6]}]
set_property IOSTANDARD LVCMOS33 [get_ports {seg[6]}]

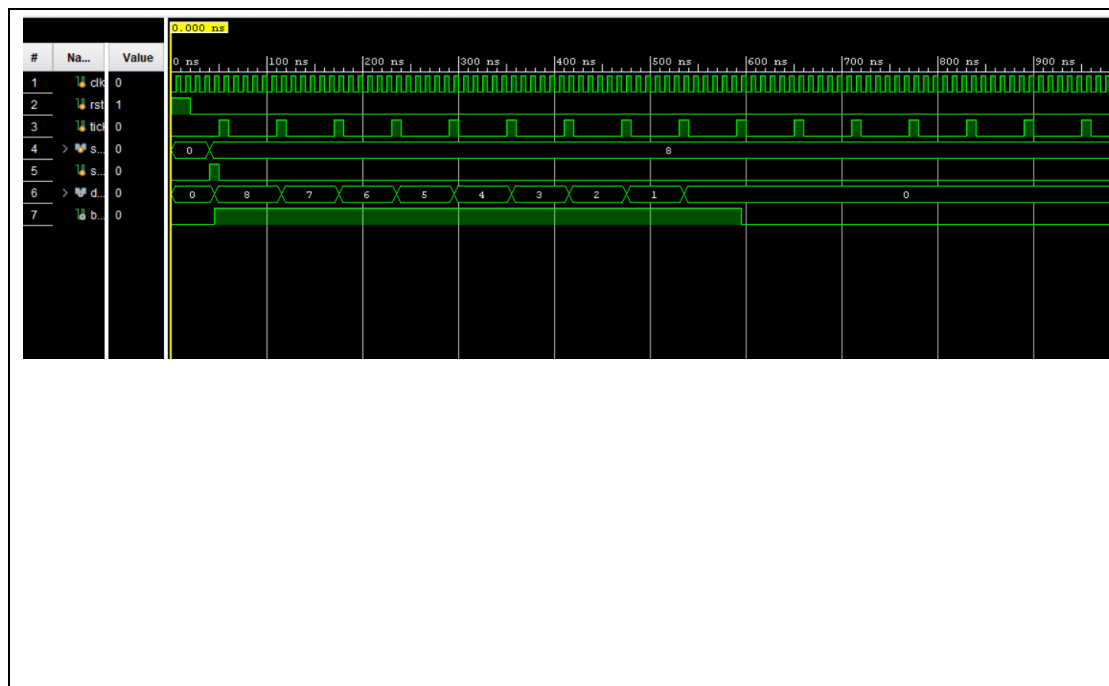
set_property PACKAGE_PIN V7 [get_ports {seg[7]}]
set_property IOSTANDARD LVCMOS33 [get_ports {seg[7]}]
```



```
## =====  
## 7-SEGMENT DIGIT ENABLES  
## =====  
  
set_property PACKAGE_PIN U2 [get_ports {an[0]}]  
set_property IOSTANDARD LVCMOS33 [get_ports {an[0]}]  
  
set_property PACKAGE_PIN U4 [get_ports {an[1]}]  
set_property IOSTANDARD LVCMOS33 [get_ports {an[1]}]  
  
set_property PACKAGE_PIN V4 [get_ports {an[2]}]  
set_property IOSTANDARD LVCMOS33 [get_ports {an[2]}]  
  
set_property PACKAGE_PIN W4 [get_ports {an[3]}]  
set_property IOSTANDARD LVCMOS33 [get_ports {an[3]}]
```

*Timing Summary*

|  |
|--|
|  |
|--|

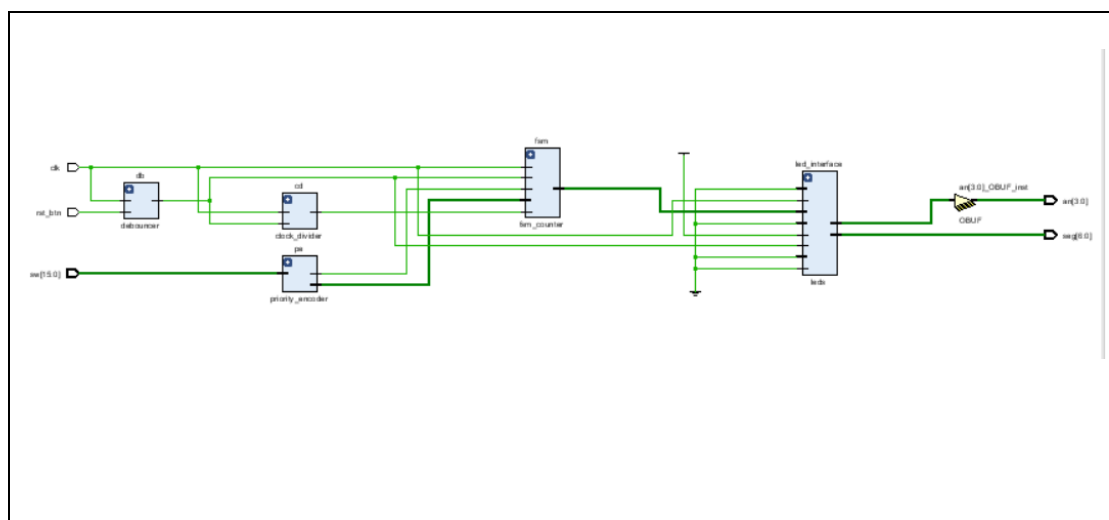


*Interpretation:*

On each rising edge of clk where tick == 1, the counter decrements by 1

## Task 4

*RTL Schematic:*



**GITHUB LINK**

<https://github.com/MohammadNaqvi1098/CALabs-Git-mn11062.git>

[https://github.com/Fakhariqblal/caLAB\\_git\\_si11091-main](https://github.com/Fakhariqblal/caLAB_git_si11091-main)

**Lab 05 – Designing FSM Using FPGA Switches and LEDs****Assessment Rubrics****Points Distribution**

| Task No. | LR2 Code | LR 5 Results | LR11 Design | AR7       |
|----------|----------|--------------|-------------|-----------|
| Prelab   |          | /15          |             | /10 & /10 |
| Task 1   | -        | -            | /10         |           |
| Task 2   | /15      | /10          |             |           |



|                     |       |     |  |     |
|---------------------|-------|-----|--|-----|
| <b>Task 3</b>       |       | /15 |  |     |
| <b>Task 4</b>       |       | /15 |  |     |
| <b>Total Points</b> | /80   |     |  | /20 |
| <b>CLO Mapped</b>   | CLO 1 |     |  |     |

*For description of different levels of the mapped rubrics, please refer the provided Lab Evaluation Assessment Rubrics and Affective Domain Assessment Rubrics.*

| #          | Assessment Elements                                   | Level 1: Unsatisfactory Points 0-1                                                                                                                                                         | Level 2: Developing Points 2                                                                                                                                                  | Level 3: Good Points 3                                                                                                                                     | Level 4: Exemplary Points 4                                                                                                                                                   |
|------------|-------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>LR2</b> | <b>Program/Code / Simulation Model/ Network Model</b> | Program/code/simulation model/network model does not implement the required functionality and has several errors. The student is not able to utilize even the basic tools of the software. | Program/code/simulation model/network model has some errors and does not produce completely accurate results. Student has limited command on the basic tools of the software. | Program/code/simulation model/network model gives correct output but not efficiently implemented or implemented by computationally complex routine.        | Program/code/simulation /network model is efficiently implemented and gives correct output. Student has full command on the basic tools of the software.                      |
| <b>LR5</b> | <b>Results &amp; Plots</b>                            | Figures/ graphs / tables are not developed or are poorly constructed with erroneous results. Titles, captions, units are not mentioned. Data is presented in an obscure manner.            | Figures, graphs and tables are drawn but contain errors. Titles, captions, units are not accurate. Data presentation is not too clear.                                        | All figures, graphs, tables are correctly drawn but contain minor errors or some of the details are missing.                                               | Figures / graphs / tables are correctly drawn and appropriate titles/captions and proper units are mentioned. Data presentation is systematic.                                |
| <b>AR9</b> | <b>Report</b>                                         | All the in-lab tasks are not included in report and / or the report is submitted too late.                                                                                                 | Most of the tasks are included in report but are not well explained. All the necessary figures / plots are not included. Report is submitted after due date.                  | Good summary of most the in-lab tasks is included in report. The work is supported by figures and plots with explanations. The report is submitted timely. | Detailed summary of the in-lab tasks is provided. All tasks are included and explained well. Data is presented clearly including all the necessary figures, plots and tables. |